BACKEND INTEGRATION RESPONSE

CSCS Upload, Reconciliation, Approval & Posting Screens

Response to the annotated frontend screen document, including the endpoint for every data panel and action, required payloads, workflow guidance, and the additions completed to support the proposed experience.

Integration status: Backend support is complete for the concerns raised.

API base path: `/api/cscs`
Authentication: Sanctum bearer token
Response date: 31 August 2026

1. Reconciliation workspace

The first screen can be populated from the following existing endpoints. Transaction groups are paginated and include their balance, flag and resolution information.

Screen area	Endpoint	Frontend guidance
Batch and summary cards	`GET /uploads/{batchId}`	Use `workflow_status`, summary fields and `allowed_actions`.
Transaction table	`GET /uploads/{batchId}/transactions?page=1&per_page=50`	Supports search, balance, flagged, resolution, security and date filters.
Selected transaction preview	`GET /uploads/{batchId}/transactions/{transactionNumber}`	Returns debit/credit legs, account effects, balance status and flag reasons.
Holding/account effect cards	`GET /uploads/{batchId}/account-effects`	Returns current, debit, credit, net and proposed quantities.
Combined preview	`GET /uploads/{batchId}/preview`	Provides batch, account effects and security mappings in one response.

> **Pagination annotation confirmed.** The proposed transaction URL is correct. Use the pagination metadata returned by Laravel rather than calculating page counts in the browser.

2. Exception list and detail panel

Load unresolved exception rows with:

```
GET /api/cscs/uploads/{batchId}/exceptions?status=UNRESOLVED&page=1&per_page=10
```

The annotation used `resolution_status=UNRESOLVED`. The API now accepts both `status` and `resolution_status` as equivalent filters, so the frontend can use either form. `status` remains the preferred documented name.

The right-hand detail panel should use the selected exception object from this paginated response. A separate request is not required unless the frontend also wants the complete transaction context; in that case, call the transaction-detail endpoint using the selected row's transaction number.

One resolution endpoint for three actions

```
POST /api/cscs/uploads/{batchId}/exceptions/{exceptionId}/resolve
```

Button	Payload
Confirm Mapping	`{ "resolution_type": "MAP_ACCOUNT", "register_account_id": 445, "reason": "Confirmed against the shareholder register." }`
Mark as Replay	`{ "resolution_type": "CONFIRM_REPLAY", "reason": "Confirmed as a previously posted movement." }`
Exclude Record	`{ "resolution_type": "RULE_EXCLUDED", "reason": "Excluded under the documented reconciliation rule." }`

`reason` is mandatory and must contain 10–1000 characters. After any resolution, refresh the exception list and call `POST /uploads/{batchId}/revalidate` before submission.

3. Creating or selecting an account

The account-related buttons use existing shareholder APIs; CSCS does not need to duplicate shareholder creation.

Action	Endpoint	Next step
Search for an account	Use the existing shareholder/register-account search endpoints.	Pass the selected register-account ID to `MAP_ACCOUNT`.
Create New Shareholder	`POST /api/shareholders`	Create the register account after receiving the shareholder ID.
Create Register Account	`POST /api/shareholders/{shareholderId}/register-accounts`	Pass the returned register-account ID to `MAP_ACCOUNT`.

> **Important:** creating a shareholder or register account does not resolve the CSCS exception by itself. The frontend must complete the sequence by calling the exception resolution endpoint.

4. Remaining reconciliation actions

Button	Endpoint	Body
Save / Revalidate	`POST /uploads/{batchId}/revalidate`	`{ "comment": "Optional reconciliation note" }`
Submit for Approval	`POST /uploads/{batchId}/submit`	`{ "comment": "Optional submission note" }`
Cancel Batch	`POST /uploads/{batchId}/cancel`	`{ "comment": "Required reason of at least 10 characters" }`

“Save & Continue Later” may simply navigate away because the staged batch and every resolution are already persisted. There is no unsaved draft payload that needs a separate endpoint.

> **Button rendering:** use `data.allowed_actions` from `GET /uploads/{batchId}`. Do not infer available actions only from the displayed status.

5. Checker review screen

Screen section	Endpoint
Metadata, totals, proposed holdings and mappings	`GET /uploads/{batchId}/preview`
Transaction groups	`GET /uploads/{batchId}/transactions`
Approval timeline	`GET /uploads/{batchId}/events` and `GET /uploads/{batchId}/approvals`
Comments and review notes	`GET /uploads/{batchId}/comments`
Post a standalone comment	`POST /uploads/{batchId}/comments` with `{ "comment": "..." }`

Checker decisions

```
POST /api/cscs/uploads/{batchId}/approve
{ "comment": "Independent review completed." }

POST /api/cscs/uploads/{batchId}/query
{ "comment": "Please confirm the source instruction reference.",
  "transaction_numbers": ["TXN-001"], "row_ids": [45] }

POST /api/cscs/uploads/{batchId}/reject
{ "comment": "Required rejection reason of at least 10 characters." }
```

6. Posting authorization

Approval and posting are intentionally separate controlled actions. Approval moves the batch to `APPROVED_AWAITING_POST`; it does not update live holdings. Only the posting action changes holdings.

Populate the confirmation modal

```
GET /api/cscs/uploads/{batchId}/posting-readiness
```

The response contains a top-level `ready` boolean, a summary of records/accounts, and individual checks for:

- approved posting state;
- unchanged immutable snapshot hash;
- active and unchanged security mappings;
- valid account mappings;
- current holdings matching the approved snapshot;
- movements not previously posted; and
- no blocking exceptions.

Disable “Authorize Posting” unless `data.ready === true` and `allowed_actions` contains `post`.

Authorize and monitor posting

```
POST /api/cscs/uploads/{batchId}/post
{ "comment": "Optional operational authorization comment." }

GET /api/cscs/uploads/{batchId}/posting-status
```

The POST returns HTTP `202 Accepted`. Poll posting status while the state is `POSTING_QUEUED` or `POSTING`. Stop polling at `POSTED`, `POSTING_FAILED` or `STALE`.

> **Role separation:** the maker cannot post their own batch. Depending on the active policy, the checker who approved it may also be prohibited from posting. Treat HTTP 403 as an authorization decision, not a retryable error.

7. Post-posting verification screen

```
GET /api/cscs/uploads/{batchId}/verification-summary
```

This consolidated response is designed for the proposed final screen. It returns:

- verification and workflow status;

- posted record and transaction-group counts;
- failed rows, debit, credit and net totals;
- updated shareholder records and created accounts;
- resolved exceptions and replay counts;
- approved totals for variance comparison; and
- the complete post-verification check result.

The UI should show success only when `verification_status` is `VERIFIED`. The backend performs verification in the same controlled transaction as posting.

8. Reports and audit downloads

The existing export endpoint now supports the four formats shown in the design:

Design card	Request
Audit Report — PDF	`GET /uploads/{batchId}/export?type=audit&format=pdf`
Posting Report — CSV	`GET /uploads/{batchId}/export?type=posting&format=csv`
Reconciliation Report — PDF	`GET /uploads/{batchId}/export?type=reconciliation&format=pdf`
Activity Log — Excel	`GET /uploads/{batchId}/export?type=activity&format=xls`

`format=xlsx` is also supported for the activity log. Existing CSV exports for rows, exceptions, reconciliation, preview and posting remain available.

9. Frontend implementation sequence

1. Load batch, preview, transactions and exceptions in parallel.
2. Render actions from `allowed_actions`.
3. Resolve exception rows using the single resolution endpoint.
4. Revalidate, ensure zero blocking exceptions, then submit.
5. On checker review, load preview, transactions, approvals, events and comments.
6. Approve, query or reject according to the returned actions.
7. Before posting, call posting readiness and display each check.
8. Authorize posting, poll posting status, then load verification summary.
9. Use the export URLs directly for report download buttons.

10. Standard handling

HTTP status	Meaning	Frontend behavior
200 / 201	Read or mutation completed	Refresh the affected batch panels.
202	Background processing accepted	Begin status polling.
403	Role or maker-checker restriction	Hide/disable action and display the returned message.
404	Batch, row or transaction not found	Return to the batch list or clear the stale selection.
422	Validation or workflow-state conflict	Show field errors; refresh batch state before retrying.

> **Conclusion:** every annotated concern now has either an existing endpoint or a newly completed endpoint. The frontend can proceed without mock-only data or unsupported action buttons.